

Assessment - Pseudo code writing

1.) Designing class hierarchy (Pseudo code)

class person

 Data

 name

 age

 Method person(name, age)

 this.name ← name

 this.age ← age

 End method

End class

class student inherits person

 Data

 registerNumber

 marks[5]

 Method student(name, age, registerNumber, marks[5])

 Super(name, age)

 this.registerNumber ← registerNumber

 this.marks ← marks

 End method

 Method calculateAverage()

 total ← 0

 For i ← 0 TO 4

 total ← total + marks[i]

 End For

 Return total/5

 End method

End class

class researchstudent inherits student

 Data

 researchArea

```
Method ResearchStudent(name, age, register  
Number, marks[], researchArea)  
Super(name, age, registerNumber, marks)  
this.researchArea ← researchArea
```

End method

```
Method displayCompleteDetail()
```

```
Print "Name: " + name
```

```
Print "Age: " + age
```

```
Print "RegisterNumber: " + registerNumber
```

```
Print "Average Marks: " + calculateAverage()
```

```
Print "ResearchArea: " + researchArea
```

End method

End class

BEGIN

```
r.s ← New ResearchStudent("Bob", 24, "R1444",  
[100, 100, 100, 100, 100], "Deep Learning")
```

```
r.s displayCompleteDetail()
```

END

2. Complete the missing Pseudo code

Begin

```
Read N
```

```
Read Array A
```

```
For i ← 0 TO N-1
```

```
For j ← i+1 TO N-1
```

```
IF A[i] == A[j]
```

```
A[j] ← -1
```

```
End IF
```

```

End For
End For
For i ← 0 To N-1
    IF A[i] ≠ -1
        Print A[i]
    End IF
End For
END

```

3. Debug the Pseudo code

```

class Area
    method area (length)
        Return length * length
    End method
    method area (length, breadth)
        Return length * breadth
    End method
End class

```

4. Trace the execution

```

BEGIN
    SUM ← 0

```

```

    FOR i ← 1 TO 5

```

```

        IF i MOD 2 == 0
            SUM ← SUM + i

```

```

        ELSE
            SUM ← SUM - i

```

```

        END IF

```

```

    END FOR

```

```

    PRINT SUM

```

```

END

```

2.)

BEGIN

READ N

READ ARRAY A

First $\leftarrow$ INFINITY

Second $\leftarrow$ INFINITY

FOR $i \leftarrow 0$ TO $N-1$

IF $A[i] > \text{First}$

Second $\leftarrow$ First

First $\leftarrow A[i]$

ELSE IF $A[i] > \text{Second}$ AND $A[i] \neq \text{First}$

Second $\leftarrow A[i]$

END IF

END FOR

IF Second == -INFINITY

PRINT "No second largest distinct
element exist"

ELSE

PRINT Second

END IF

END

6. Convert Problem Statement to Pseudo code

INTERFACE PaymentMethod

METHOD pay(amount)

END INTERFACE

CLASS CreditCard IMPLEMENT PaymentMethod

METHOD pay(amount)


```
PRINT "PAID" + amount + "using credit card"
```

```
END METHOD
```

```
END CLASS
```

```
CLASS UPI IMPLEMENTS PaymentMethod
```

```
METHOD pay(amount)
```

```
PRINT "paid" + amount + "using UPI"
```

```
END METHOD
```

```
END CLASS
```

```
CLASS NetBanking IMPLEMENTS PaymentMethod
```

```
METHOD pay(amount)
```

```
PRINT "paid" + amount + "using netbanking"
```

```
END METHOD
```

```
END CLASS
```

```
BEGIN
```

```
READ choice
```

```
READ amount
```

```
PaymentMethod method
```

```
IF choice == "credit card"
```

```
method ← New CreditCard()
```

```
ELSE IF choice == "UPI"
```

```
method ← NewUPI()
```

```
ELSE IF choice == "net banking"
```

```
method ← New NetBanking()
```

```
END IF
```

```
method.pay(amount)
```

```
END
```

7. Runtime Polymorphism

```
CLASS Animal
```

```
    METHOD speak()
```

```
        PRINT "Animal"
```

```
    END METHOD
```

```
END CLASS
```

```
CLASS Dog INHERITS Animal
```

```
    METHOD speak()
```

```
        PRINT "BARK"
```

```
    END METHOD
```

```
END CLASS
```

```
Animal obj ← New Dog()
```

```
obj.speak()
```

output: BARK

8. Find logical errors (consider)

```
CLASS Employee
```

```
    DATA
```

```
        id
```

```
        name
```

```
    CONSTRUCTOR (empid, empname)
```

```
        id ← empid
```

```
        name ← empname
```

```
    END CONSTRUCTOR
```

```
END CLASS
```

```
BEGIN
```

```
    c ← new NEW Employee (10, "Ram")
```

```
END
```

9. Case study : vehicle management

CLASS vehicle

DATA

vehicleNo

Owner Name

METHOD vehicle (vehicleNo, ownerName)

this. vehicleNo $\leftarrow$ vehicleNo

this. ownerName $\leftarrow$ ownerName

END METHOD

METHOD display()

PRINT "vehicle no: " + vehicleNo

PRINT "Owner Name: " + ownerName

END METHOD

END CLASS

CLASS car INHERITS vehicle

DATA

fuel type

Method car (vehicle no, owner name, fuel type)

SUPER (vehicle no, ownerName)

this. fuelType $\leftarrow$ fuel type

END METHOD

END CLASS

CLASS truck INHERITS vehicle

DATA

load capacity

METHOD truck (vehicle no, ownerName,
load capacity)

Super (vehicle no, ownerName)

this. load capacity $\leftarrow$ load capacity

END METHOD

METHOD display()

SUPER.display()

PRINT "Load capacity: " + loadCapacity

END METHOD

END CLASS

BEGIN

READ N

vehicle[0] ← N

FOR i ← 0 TO N-1

 READ type

 READ vehicleNo, ownerName

 IF type == "car"

 READ fuelType

 vehicle[i] ← new Car(vehicleNo,
 ownerName, fuelType)

 ELSE IF type == "truck"

 READ loadCapacity

 vehicle[i] ← new Truck(vehicleNo,
 ownerName, loadCapacity)

 END IF

END FOR

FOR i ← 0 TO N-1

 vehicle[i].display()

END FOR

END